

Lab 1: Implementing a class

24292-Object Oriented Programming

1 Introduction

The aim of this lab is to implement the design of Seminar 1. The session is mandatory and you have to deliver the source code of the Java project and a document describing the implementation. Concretely, we will implement the two classes `Vector` and `Record` from Seminar 1, as well as a class `TestRecord` for testing these classes.

You are free to use implement your own solution from Seminar 1, but for reference, a sample solution appears below.

Vector	Record
-elems: array of double	-input: Vector -output: double
+Vector(e:array of double) +Vector(dim:int,val:double) +getDim(): int +add(v:Vector): Vector +subtract(v:Vector): Vector +multiply(v:Vector): Vector +divide(v:Vector): Vector +multiply(scalar:double): Vector +divide(scalar:double): Vector +sqrt(): Vector +dotProduct(v:Vector): double +norm(): double +toString(): String	+Record(i:Vector,o:double) +getInput(): Vector +getOutput(): double +toString(): String

2 The TestRecord class

First implement a class called `TestRecord` with a main method for testing the other implemented classes. Even after being implemented in a `.java` file, a class definition does nothing on its own. It is only when you create instances of a class and call methods on these instances that you can observe what the methods do. In this sense, the main method is the most important method, since this is where all the other code is tested.

During the implementation of a given class definition, it is a good idea to frequently test the class in the main method by creating instances and calling methods on them. This is the only way to make sure that the methods have been correctly implemented. After writing a class definition such as `Vector` you should *always* test all of its methods. In addition, if you write a lot of code without testing, it becomes more difficult to locate the source of an error.

2.1 The Vector class

Next implement the `Vector` class. Add the attributes and methods as specified. The sample solution uses an *array* to store the elements of the vector, but you are free to use another type of collection. The sample solution also has two constructors: one that takes an already existing array as input, and another that creates a new vector with a given dimensionality and sets all elements equal to `val`. The getter `getDim` returns the dimensionality of the vector. All other methods are described in the handout of Seminar 1.

If you decide to use an array to store the elements of the vector, the existing Java class `Arrays` contains several useful operations on arrays. Test the class by creating several vectors, calling each method and printing the result to check that everything works correctly. The reason for the `toString` method is precisely to have a convenient way to print the content of a vector to the screen. Common mathematical operations such as the square root and absolute value are implemented as part of the existing Java class `Math`.

2.2 The Record class

Finally implement the `Record` class by adding the corresponding attributes and methods. This class is simpler than `Vector`, and in addition to the attributes, there is a constructor method, two getter methods and a method `toString` for convenient printing.

2.3 Optional: more compact methods of Vector

Several of the methods of `Vector` have an almost identical structure: create a new instance of `Vector` and compute the elements by applying a unary or binary function to the elements of one or two existing vectors. The only thing that changes is the concrete unary or binary function used to combine elements: addition, subtraction, square root, etc. Optionally, try to make the methods of `Vector` more compact by writing a single base method for creating a vector using a unary operation, and another base method for creating a vector using a binary operation. The individual methods can now be implemented using lambda calculus, which you have to investigate on your own for Java. (This is a rather difficult optional task mostly intended for highly motivated students.)

3 Documentation

Apart from the source code, you should also upload to your Git repository a document that outlines the solution of the problem. To elaborate the document you can use the following guidelines regarding the content:

1. An introduction where the problem is described. For example, what should the program do? Which classes do you have to define? Which methods do you have to implement for these classes?
2. A description of possible alternative solutions that were discussed, and a description of the chosen solution and the reason for choosing this solution rather than others. It is also a good idea to mention the related theoretical concepts of object-oriented programming that were applied as part of the solution.
3. A conclusion that describes how well the solution worked in practice, i.e. did the tests show that the classes were correctly implemented? You can also mention any difficulties during the implementation as well as any doubts you might have had.

4 Submission

The source code and documentation should all be uploaded to a directory **Lab1** of your Git repository **prior to the next lab**. Please make sure that the directory name is *exactly* **Lab1** (not **Lab_1**, **Lab-1**, **LAB1**, **lab1** etc.), and that **Lab1** is located in the **root of the Git repository**. Failure to comply with these rules will result in a grade deduction.